

39

Streamlit

Turn Python scripts into web apps, fast
Show off your AI projects with no web development

[AI/ML Engineer Learning Series · Deep Dive Edition](#)

What it is	A tool to build web apps in pure Python
No need for	HTML, CSS, JavaScript, or React
Great for	Demos of your ML and AI projects
The win	A working app in minutes, not weeks
Stage	The start of deployment — showing your work

What's Inside

Here is everything covered in this guide, in order:

#	Section	What you'll learn
1	What is Streamlit?	Python to web app
2	Why It's Perfect for AI	Demos without web dev
3	Your First App	Three lines to a web page
4	The Main Widgets	Buttons, sliders, inputs
5	How Streamlit Works	The re-run model
6	Session State	Remembering between re-runs
7	A Real ML App	Wrapping your model
8	Sharing Your App	Getting it online
9	Common Mistakes	What to avoid
★	Cheatsheet	Quick reference

1. What is Streamlit?

Streamlit is a Python library that turns a normal Python script into an interactive web app. You write plain Python — no HTML, CSS, or JavaScript — and Streamlit builds the web page for you, with buttons, sliders, charts, and text. It's the fastest way to put a friendly face on your code.



Fig 1 — Streamlit turns your Python script into a real, interactive web app — no web development needed.

This starts the deployment stage of your journey: getting your projects out of notebooks and into apps people can actually use. After building all those models, Streamlit is how you show them off — a clickable demo beats a code file every time, especially for your portfolio and job applications.

■ *You have web dev experience (MERN), so you know how much work a real web app is. Streamlit's appeal is skipping all of that when you just need a demo — pure Python, working in minutes.*

2. Why It's Perfect for AI

Building a normal web app for your model means HTML, CSS, JavaScript, a backend, and server setup — a lot of work just to show a demo. Streamlit skips all of it. You stay in Python, the language your model is already written in.

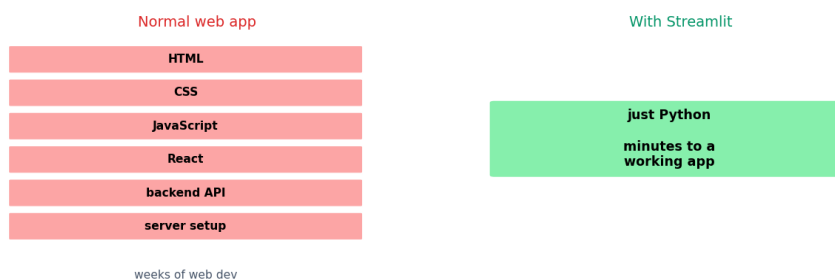


Fig 2 — A normal web app needs the full web stack. Streamlit needs only Python.

For data scientists and ML engineers, this is a perfect match. Your model, your data processing, and your app are all in one Python file. You can build a working demo of your Visual Defect Detector or RAG chatbot in an afternoon, then share a link. That's huge for a portfolio.

3. Your First App

A Streamlit app is just a Python script using the streamlit library (written as st). Here's a complete app:

```
# pip install streamlit
# save as app.py
import streamlit as st

st.title('My First App')
st.write('Hello! This is a web app made with Python.')

name = st.text_input('What is your name?')
if name:
    st.write(f'Nice to meet you, {name}!')
```

Run it from your terminal with: **streamlit run app.py**. A web page opens in your browser with a title, text, and a working input box. That's a real, interactive web app in about six lines of Python. No other setup.

■ *Every Streamlit command starts with st. — st.title, st.write, st.button, and so on. Once you know a handful of these, you can build most apps. It's a small, friendly toolkit.*

4. The Main Widgets

Widgets are the interactive pieces — the buttons, sliders, and inputs users click and type into. Streamlit has one for almost everything, each a single line.

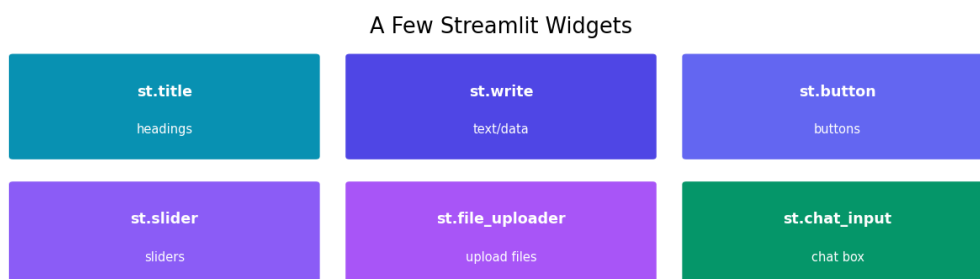


Fig 3 — A few common Streamlit widgets, each created with a single line of Python.

Widget	What it does
st.title / st.header	Headings
st.write	Show text, data, charts — almost anything
st.button	A clickable button
st.slider	A slider to pick a number
st.text_input	A text box for typing
st.file_uploader	Let users upload a file (image, PDF)
st.chat_input	A chat box — perfect for your RAG chatbot

5. How Streamlit Works

Here's the one quirk to understand. Every time a user interacts — clicks a button, moves a slider, types — Streamlit re-runs your ENTIRE script from top to bottom. It then shows the updated result. This 're-run on every action' model is simple but important to know.

How Streamlit Works: re-run on every action



Streamlit re-runs the whole script on every interaction

Fig 4 — On every interaction, Streamlit re-runs the whole script and refreshes the page.

This makes Streamlit easy to reason about — the page always reflects your current code, top to bottom. But it has a catch: anything slow (like loading a big model) would re-run every time too, which would be painfully slow. The fix is caching, which you'll want for any real ML app.

```
# Cache slow work so it only runs ONCE, not every re-run
@st.cache_resource
def load_model():
    return load_my_big_model() # runs once, then reused

model = load_model() # fast on every re-run after the first
```

■ *@st.cache_resource is essential for ML apps. Without it, your model would reload on every click — unbearably slow. With it, the model loads once and stays ready. Always cache model loading.*

6. Session State

Because the script re-runs each time, normal variables reset on every interaction. To remember things across re-runs — like a chat history — Streamlit gives you `session_state`, a place that survives between runs.

```
# Keep chat history across re-runs
if 'messages' not in st.session_state:
    st.session_state.messages = []

# Add to it (it persists)
if prompt := st.chat_input('Ask me anything'):
    st.session_state.messages.append(prompt)

# Show all messages
for msg in st.session_state.messages:
    st.write(msg)
```

`session_state` is how you build a working chatbot interface — it holds the conversation so it doesn't vanish on each new message. For your RAG chatbot's UI, this is exactly what you'd use.

7. A Real ML App

Here's the shape of a real app wrapping an image classifier — like your Product Image Classifier or Defect Detector. Upload an image, get a prediction.

```
import streamlit as st
from PIL import Image

st.title('Image Classifier Demo')

@st.cache_resource
def load_model():
    return load_my_model() # cached — loads once

model = load_model()

# Let the user upload an image
file = st.file_uploader('Upload an image', type=['jpg', 'png'])

if file:
    image = Image.open(file)
    st.image(image, caption='Your image')

    if st.button('Predict'):
        label, confidence = model.predict(image)
        st.success(f'Prediction: {label} ({confidence:.0%})')
```

■ *This is your portfolio in action: wrap any of your models in ~20 lines of Streamlit and suddenly it's a clickable demo recruiters can try. Far more impressive than a notebook.*

8. Sharing Your App

An app on your computer is good; an app others can open with a link is great. There are easy ways to put a Streamlit app online for free.

Option	Notes
Streamlit Community Cloud	Free, official, connects to your GitHub
Hugging Face Spaces	Free, popular for ML demos
Render / Railway	Free tiers, more control

Streamlit Community Cloud is the easiest: push your app to GitHub, connect it, and you get a public link in minutes — free. Hugging Face Spaces (which you'll see again at the end of this stage) is also excellent for ML demos. Either way, you end up with a shareable link to put on your portfolio and CV.

■ *A live, clickable demo link on your CV is a strong signal to employers. It shows you don't just train models — you can ship them as working products. This is exactly the gap deployment skills fill.*

9. Common Mistakes

Mistake	Why it's a problem	Fix
Not caching the model	Reloads every click, very slow	Use <code>@st.cache_resource</code>
Expecting variables to persist	They reset each re-run	Use <code>st.session_state</code>
Heavy work outside cache	Slow app	Cache slow functions
Forgetting 'streamlit run'	Script won't show as app	Run with <code>streamlit run app.py</code>
Huge model on free hosting	Runs out of memory	Use smaller models for demos
No error handling on upload	Crashes on bad files	Check the uploaded file first

Quick Reference Cheatsheet

Task	Code
Install	<code>pip install streamlit</code>
Run an app	<code>streamlit run app.py</code>
Import	<code>import streamlit as st</code>
Title / heading	<code>st.title('...') / st.header('...')</code>
Show anything	<code>st.write(x)</code>
Text input	<code>st.text_input('label')</code>
Button	<code>st.button('Click me')</code>
Slider	<code>st.slider('label', 0, 100)</code>
Upload a file	<code>st.file_uploader('label')</code>
Show an image	<code>st.image(img)</code>
Chat input	<code>st.chat_input('Ask...')</code>
Cache a model	<code>@st.cache_resource</code> above the loader
Remember state	<code>st.session_state</code>
Success message	<code>st.success('done!')</code>

The one idea to remember: Streamlit turns a Python script into an interactive web app with no web development. Write `st.something` commands, run with `streamlit run`, and your model becomes a clickable demo — the fastest way to share your AI projects.

Next Topic: Gradio — another simple way to build AI demos, especially loved in the ML community and tightly connected to Hugging Face Spaces. We'll see how it compares to Streamlit.